

Podsumowanie Prac CUDSS

W ramach projektu udało się uruchomić bibliotekę NVIDIA cuDSS zarówno lokalnie, jak i w środowisku chmurowym.

Kod aplikacji został umieszczony w repozytorium GitHub:
https://github.com/filipdukat/cudss_sparse_solver

W środowisku lokalnym wykorzystano kartę graficzną NVIDIA GeForce RTX 3050 Laptop GPU wyposażoną w 4 GB pamięci VRAM oraz 16 GB pamięci RAM systemowej. Następnie projekt został przeniesiony do środowiska chmurowego RunPod.io, gdzie wykorzystano serwer wyposażony w GPU NVIDIA RTX PRO 4500 posiadające 24 GB pamięci VRAM, 62 GB pamięci RAM oraz 28 vCPU.

Zaimplementowano mechanizm wczytywania dużych macierzy rzadkich zapisanych w binarnym formacie COO (Coordinate Format). Program odczytuje następujące parametry macierzy:

- liczbę wierszy,
- liczbę kolumn,
- liczbę elementów niezerowych (NNZ),
- informację o liczbach zespolonych.

Następnie zaimplementowano konwersję z formatu COO do CSR wymaganą przez bibliotekę NVIDIA cuDSS.

Dodatkowo zaimplementowano obsługę wektora prawej strony układu równań liniowych (RHS vector) b , również zapisanego w binarnym formacie. Format odczytu został zaimplementowany na podstawie informacji zawartych w dokumentacji projektu (pdf na dysku udostępniony przez opiekuna projektu), zgodnie z następującym schematem zapisu:

- zapis długości wektora jako uint64,
- zapis flagi określającej liczby zespolone,
- zapis danych typu double,
- w przypadku liczb zespolonych osobny zapis części rzeczywistej i urojonej.

W pierwszym etapie wykonano pełny pipeline biblioteki cuDSS dla środowiska lokalnego obejmujący:

- analysis,
- factorization,
- solve.

Test został przeprowadzony dla macierzy o następujących parametrach:

- liczba wierszy: 472 096,
- liczba elementów niezerowych (NNZ): 17 592 472,

- liczba niezerowych elementów wektora b : 437.

Solver poprawnie wykonał:

- alokację pamięci GPU,
- kopiowanie danych,
- analizę struktury macierzy,
- faktoryzację,
- rozwiązanie układu równań.

W wyniku działania solvera otrzymano poprawny wektor rozwiązania x zawierający stabilne numerycznie wartości zmiennoprzecinkowe.

```
Loading COO matrix...
Converting COO -> CSR...
Loading RHS vector...
Vector size: 472096
isComplex: 0
nonzero b[60] = 1.4037231402
nonzero b[61] = 1.2264463014
nonzero b[228] = -1.3456898137
nonzero b[229] = -1.2555013257
nonzero b[902] = 1.7451716045
nonzero b[903] = 1.1732871696
nonzero b[904] = 1.8387848537
nonzero b[905] = 1.2509337720
nonzero b[906] = 1.4916580126
nonzero b[907] = 1.1390782477
Total nonzero b values: 437
Rows: 472096
NNZ : 17592472
Allocating GPU memory...
Copying data to GPU...
Running analysis...
Running factorization...
Running solve...
=====
SOLVE FINISHED
=====
First 10 x values:
x[0] = 0.0243214209
x[1] = -0.0010710765
x[2] = 0.0009010353
x[3] = 0.0025117503
x[4] = 0.0043860996
x[5] = -0.0026682566
x[6] = 0.0023424761
x[7] = 0.0032004068
x[8] = 0.0616945124
x[9] = 0.0794911330
```

Rys. 1 przedstawia wynik działania programu w środowisku lokalnym dla macierzy o liczbie wierszy wynoszącej 472 096.

W kolejnym etapie podjęto próbę rozwiązania znacznie większego układu równań lokalnie na GPU RTX 3050. Wykorzystano macierz o parametrach:

- liczba wierszy: 1 432 119,
- liczba elementów niezerowych (NNZ): 107 043 349,
- liczba niezerowych elementów wektora b : 1099.

Podczas etapu factorization wystąpił błąd biblioteki cuDSS:
„CUDSS ERROR: factorization”.

Problem wynikał z ograniczonych zasobów sprzętowych lokalnego GPU, przede wszystkim niewystarczającej ilości pamięci VRAM wymaganej do przeprowadzenia faktoryzacji bardzo dużej macierzy sparse.

```

=====
cuDSS Sparse Solver
=====
Loading COO matrix...
Converting COO -> CSR...
Loading RHS vector...
Vector size: 1432119
isComplex: 0
nonzero b[230] = 1.4250954353
nonzero b[231] = 1.3015408949
nonzero b[295] = -1.2995078180
nonzero b[296] = -1.2035596094
nonzero b[5597] = 1.5647116548
nonzero b[5598] = 1.1981966031
nonzero b[5600] = 1.7843628104
nonzero b[5601] = 1.0697386599
nonzero b[5602] = 1.7768972228
nonzero b[5603] = 1.5555937827
Total nonzero b values: 1099
Rows: 1432119
NNZ : 107043349
Allocating GPU memory...
Copying data to GPU...
Running analysis...
Running factorization...
CUDSS ERROR: factorization

```

Rys. 2 przedstawia wynik działania programu w środowisku lokalnym dla macierzy o liczbie wierszy wynoszącej 1 432 119.

W związku z tym zdecydowano się na wykorzystanie środowiska chmurowego RunPod.io. Utworzono środowisko Linux z wykorzystaniem GPU NVIDIA RTX PRO 4500 oraz skonfigurowano:

- CUDA Toolkit,

- NVIDIA cuDSS,
- system budowania CMake.

Po poprawnym załadowaniu danych wykonano pełny pipeline biblioteki cuDSS:

- analysis,
- factorization,
- solve.

W efekcie rozwiązano układ równań liniowych:

$$Ax = b$$

dla bardzo dużej macierzy sparse o parametrach:

- liczba wierszy: 1 432 119,
- liczba elementów niezerowych (NNZ): 107 043 349.
- liczba niezerowych elementów wektora b: 1099.

Solver poprawnie wyznaczył wektor rozwiązania x, a otrzymane wartości miały stabilny charakter numeryczny. Nie wystąpiły błędy typu overflow, NaN ani nierealistycznie duże wartości numeryczne.

```

=====
cuDSS Sparse Solver
=====
Loading COO matrix...
Converting COO -> CSR...
Loading RHS vector...
Vector size: 1432119
isComplex: 0
nonzero b[230] = 1.4250954353
nonzero b[231] = 1.3015408949
nonzero b[295] = -1.2995078180
nonzero b[296] = -1.2035596094
nonzero b[5597] = 1.5647116548
nonzero b[5598] = 1.1981966031
nonzero b[5600] = 1.7843628104
nonzero b[5601] = 1.0697386599
nonzero b[5602] = 1.7768972228
nonzero b[5603] = 1.5555937827
Total nonzero b values: 1099
Rows: 1432119
NNZ : 107043349
Allocating GPU memory...
Copying data to GPU...
Running analysis...
Running factorization...
Running solve...
=====
SOLVE FINISHED
=====
First 10 x values:
x[0] = -0.0382466308
x[1] = -0.0052125733
x[2] = -0.0126186966
x[3] = -0.0877277121
x[4] = -0.0988278185
x[5] = -0.0621376332
x[6] = -0.0841807069
x[7] = -0.0871313687
x[8] = -0.0844951717
x[9] = -0.1171131090

```

Rys. 3 przedstawia wynik działania programu w środowisku chmurowym dla macierzy o liczbie wierszy wynoszącej 1 432 119.

Uzyskano działający pipeline umożliwiający dalsze testy wydajnościowe oraz przyszłe porównania z innymi bibliotekami solverów sparse wykorzystywanymi w projekcie, takimi jak STRUMPACK, Intel oneAPI PARDISO oraz PaNua.